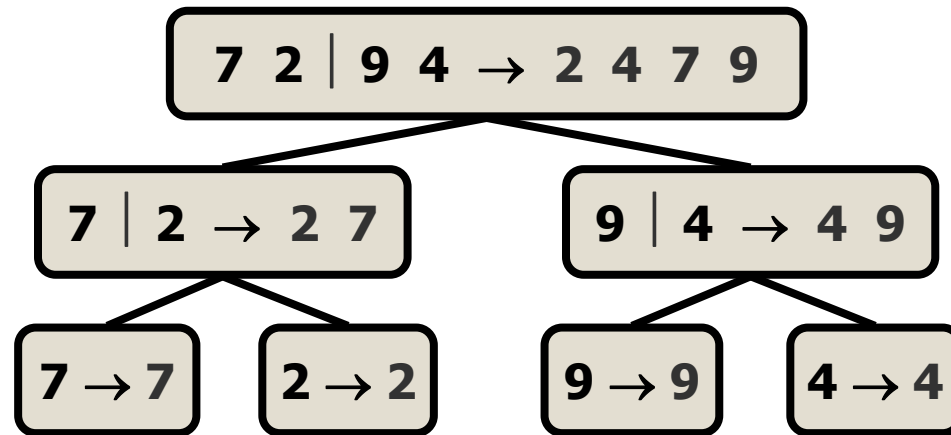


Lecture 06 Sorting and Selection Algorithms



Learning Outcome

- ◆ Merge-sort (4.1.1)
- ◆ Summary of Sorting Algorithms
- ◆ Bucket-Sort (4.5.1)
- ◆ Radix-Sort (4.5.2)
- ◆ Quick-Select (§ 4.7)

In-Place Algorithms

- ◆ An in-place algorithm produces its result by overwriting the input data structure. It does not require an additional temporary data structure to store the result.
- ◆ A not-in-place algorithm writes its temporary result to an additional temporary data structure, use the temporary result to store the result, then copy the result back to the original data structure.
- ◆ All sorting algorithms studied before this chapter are in-place. They do not need a separate array to store result.

Merge-Sort

- ◆ Merge-sort is a sorting algorithm based on the divide-and-conquer paradigm
- ◆ Like heap-sort
 - It has $O(n \log n)$ running time
- ◆ Unlike heap-sort
 - It accesses data in a sequential manner (suitable to sort data on a disk)
 - It is stable
 - It is not-in-place

Merge-Sort (cont.)

- ◆ Merge-sort on an input sequence S with n elements consists of 3 steps:
 - Divide: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
 - Recur: recursively sort S_1 and S_2
 - Conquer: merge S_1 and S_2 into a sorted sequence

Algorithm Merge-Sort

Algorithm MergeSort (S, left, right)

Input: array S, left index, right index.

Output: array S sorted.

// continue if has more than one element.

if left < right

 // Divide: divide S into two nearly equal parts.

 mid = (left + right) / 2

 // Recur: sort each half.

 MergeSort (S, left, mid) // left half, s1

 MergeSort (S, mid + 1, right) // right half, s2

 // Conquer: merge the two halves into a sorted S.

 Merge (S, left, mid, right)

Merging Two Sorted Sequences

- ◆ The conquer step of merge-sort consists of merging two sorted sequences L and R into a sorted sequence A[left]...A[right]
- ◆ Merging two sorted sequences, each with $n/2$ elements, takes $O(n)$ time

```
Algorithm Merge (S, left, mid, right)
  Input: array S, left index, mid index,
         right index.
  Output: sorted array S[left]...S[right].

  L = new LinkedList from S[left]...S[mid]
  R = new LinkedList from S[mid+1]...S[right]

  k = left
  while not L.isEmpty() and not R.isEmpty()
    if L.first() < R.first()
      S[k++] = L.removeFirst()
    else
      S[k++] = R.removeFirst()
  while not L.isEmpty()
    S[k++] = L.removeFirst()
  while not R.isEmpty()
    S[k++] = R.removeFirst()
```

Merging Two Sorted Sequences (cont.)

- ◆ Implementation of Merge-Sort that is performance efficient requires a temporary data structure (L and R) in the Conquer step, making Merge-Sort an not-in-place algorithm.

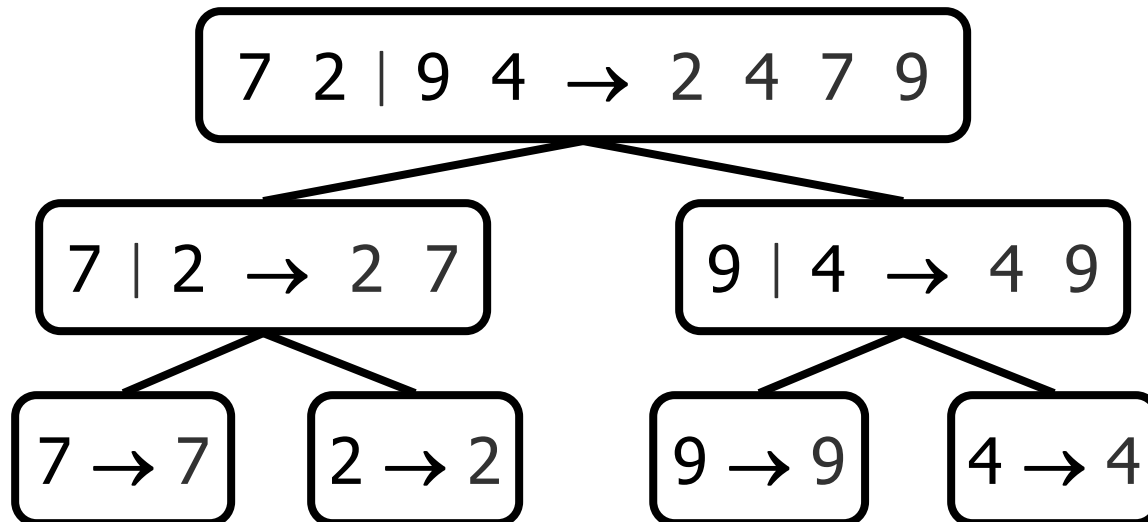
```
Algorithm Merge (S, left, mid, right)
  Input: array S, left index, mid index,
         right index.
  Output: sorted array S[left]...S[right].

  L = new LinkedList from S[left]...S[mid]
  R = new LinkedList from S[mid+1]...S[right]

  k = left
  while not L.isEmpty() and not R.isEmpty()
    if L.first() < R.first()
      S[k++] = L.removeFirst()
    else
      S[k++] = R.removeFirst()
  while not L.isEmpty()
    S[k++] = L.removeFirst()
  while not R.isEmpty()
    S[k++] = R.removeFirst()
```


Merge-Sort Tree

- ◆ An execution of merge-sort is depicted by a binary tree
 - each node represents a recursive call of merge-sort and stores
 - ◆ unsorted sequence before the execution and its partition
 - ◆ sorted sequence at the end of the execution
 - the root is the initial call
 - the leaves are calls on subsequences of size 0 or 1



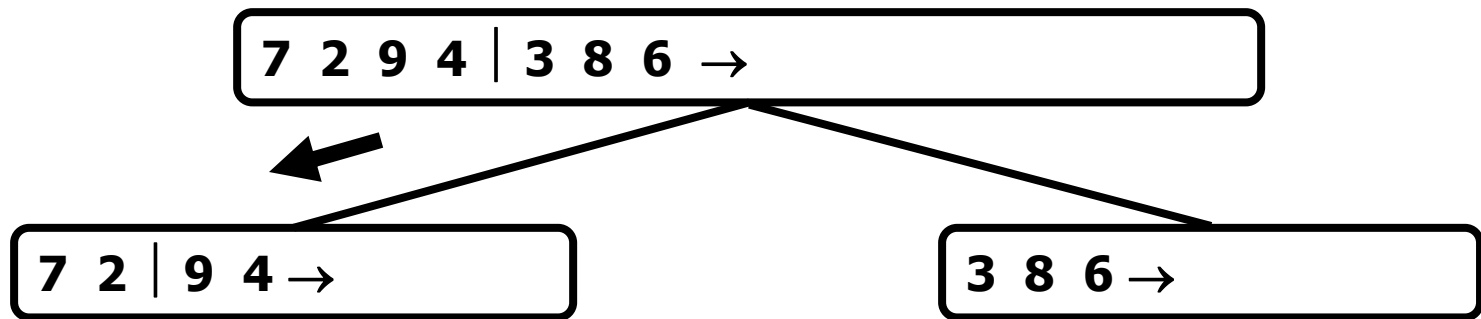
Merge-Sort Tree Example

◆ Divide (Why not 7 2 9 | 4 3 8 6?)

7 2 9 4 | 3 8 6 →

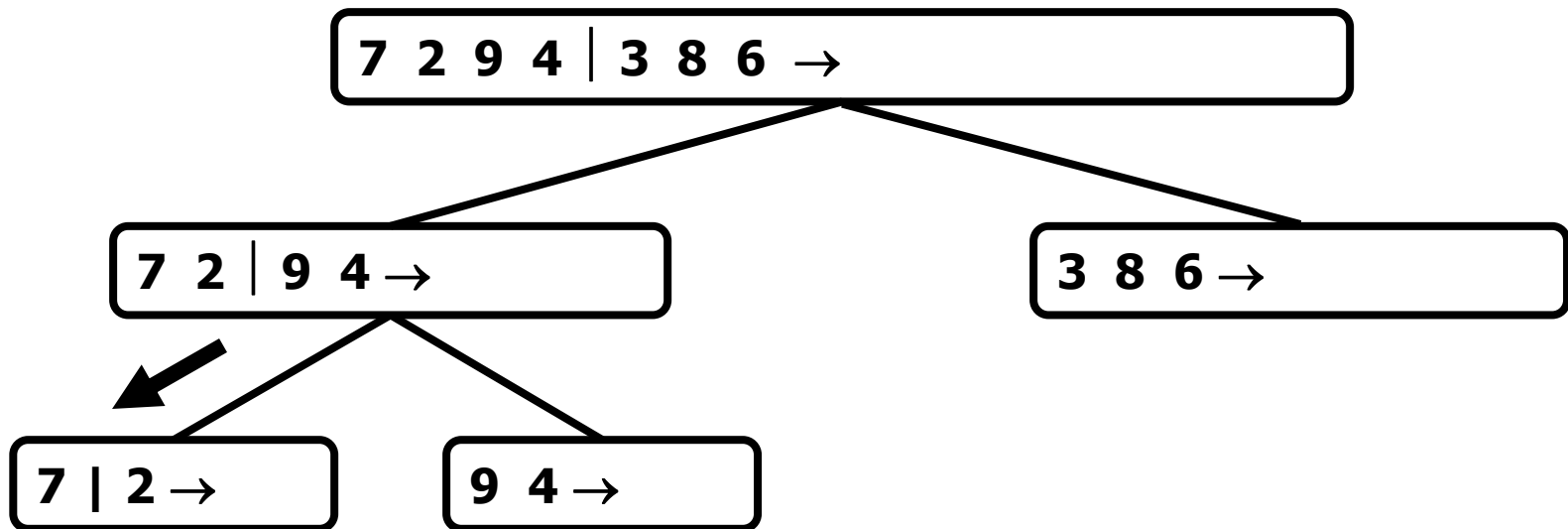
Merge-Sort Tree Example (cont.)

◆ Recursive left, divide



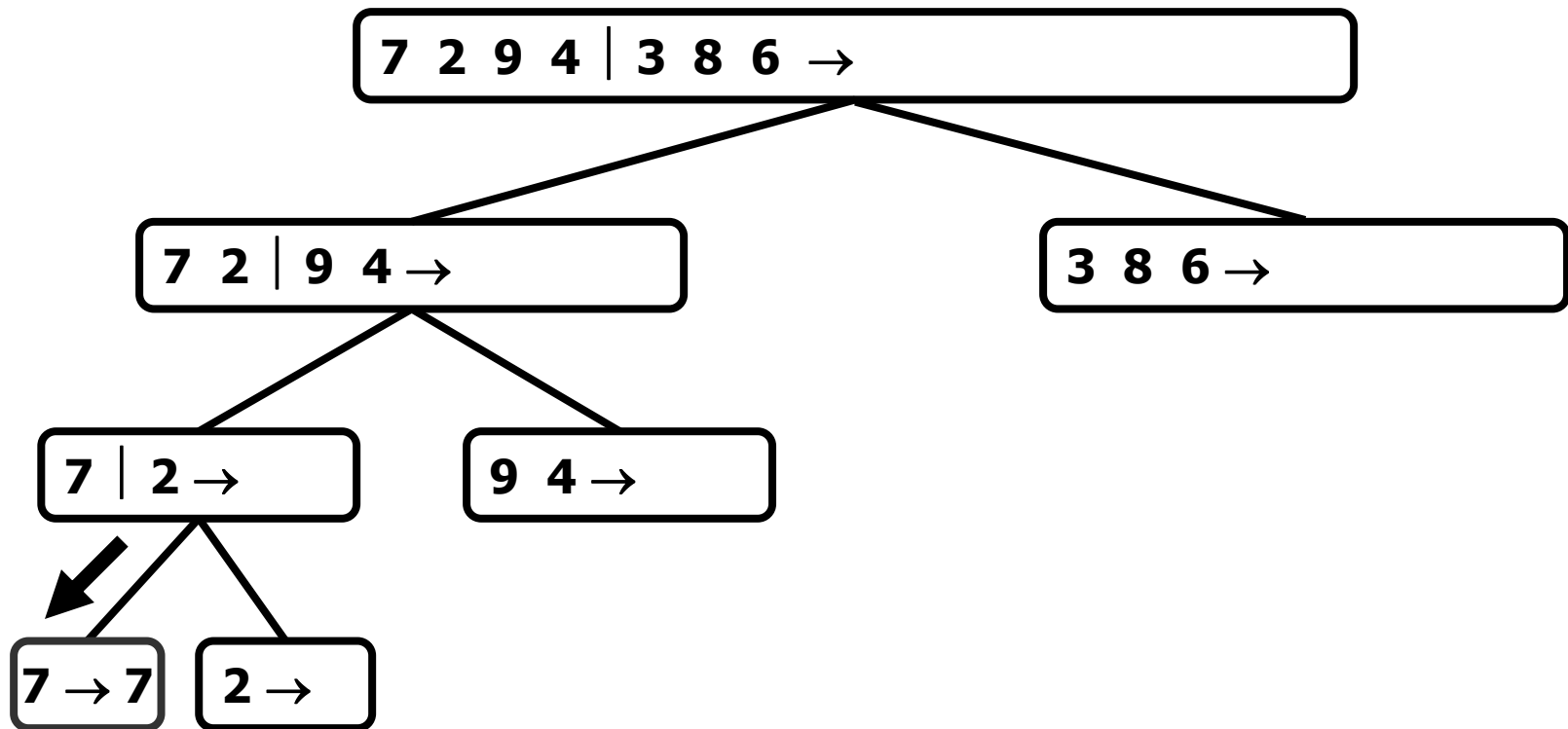
Merge-Sort Tree Example (cont.)

◆ Recursive left, divide



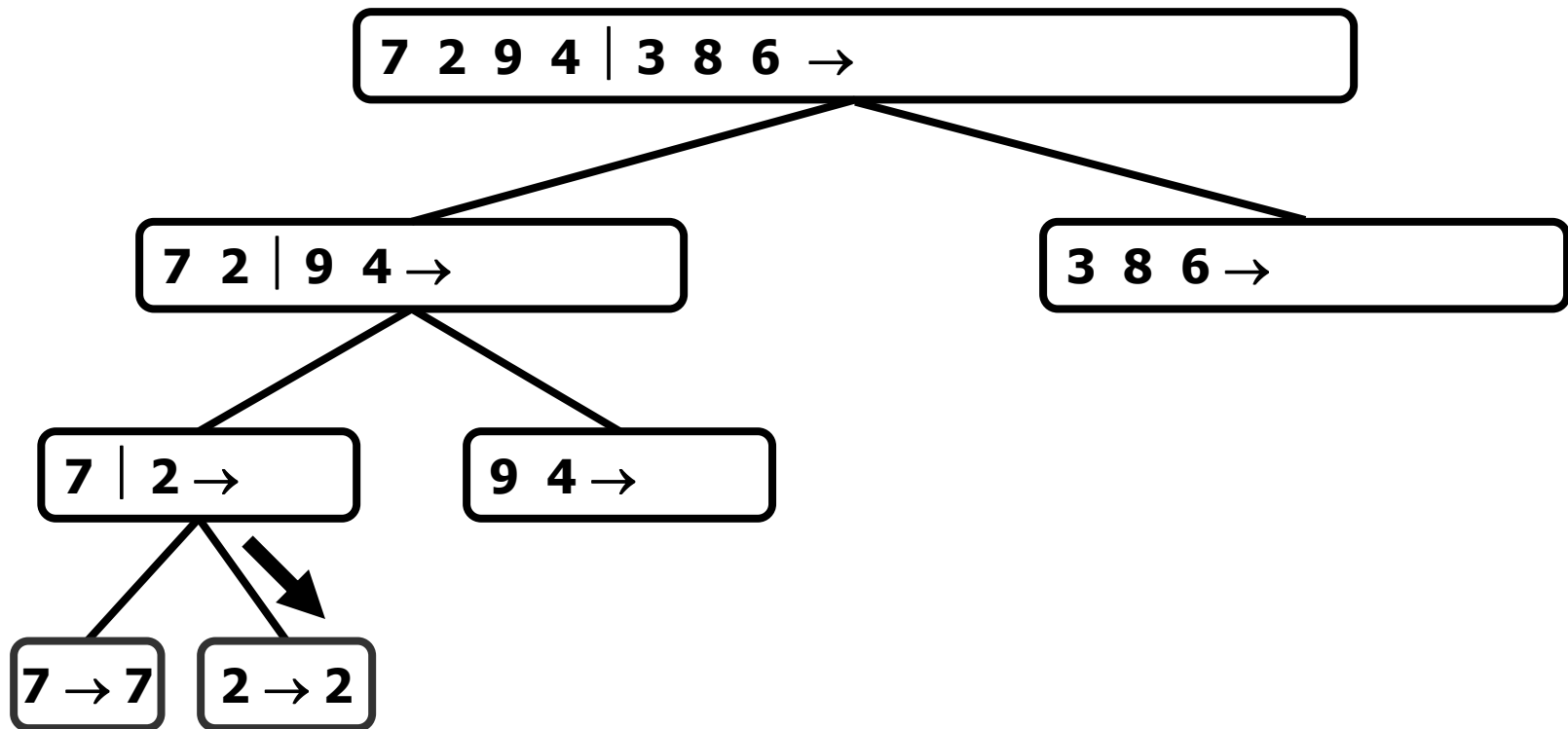
Merge-Sort Tree Example (cont.)

◆ Recursive left, base case



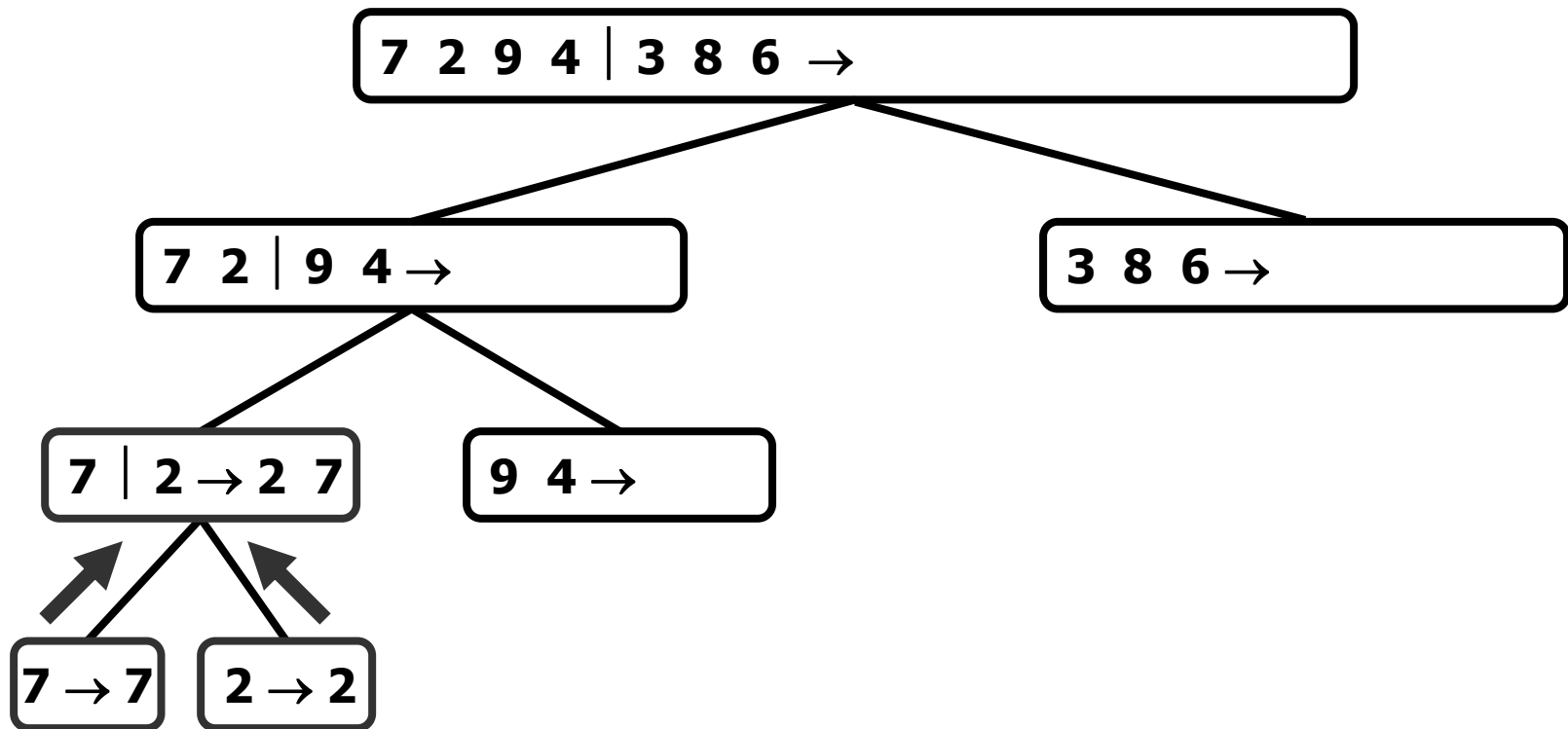
Merge-Sort Tree Example (cont.)

◆ Recursive right, base case



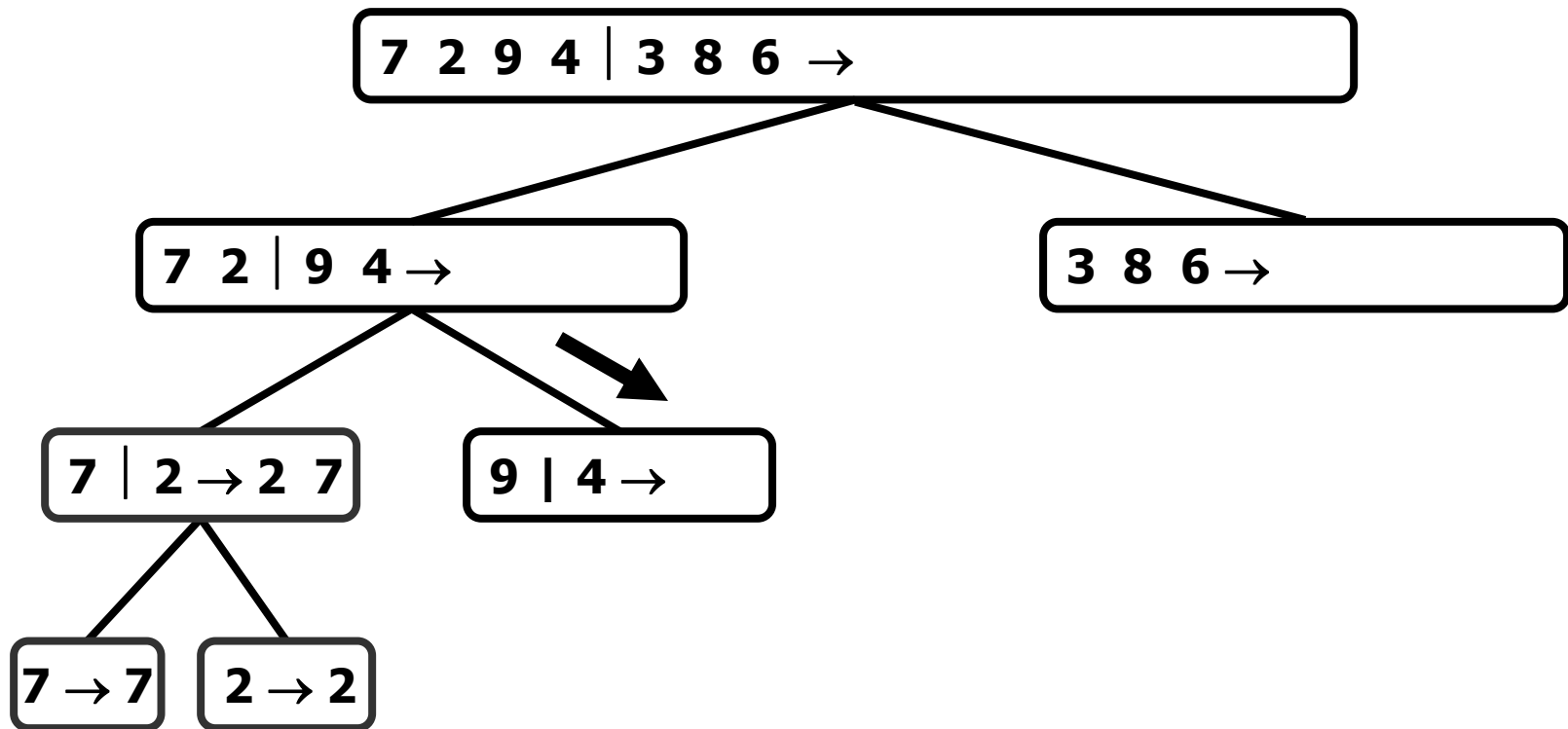
Merge-Sort Tree Example (cont.)

◆ Merge



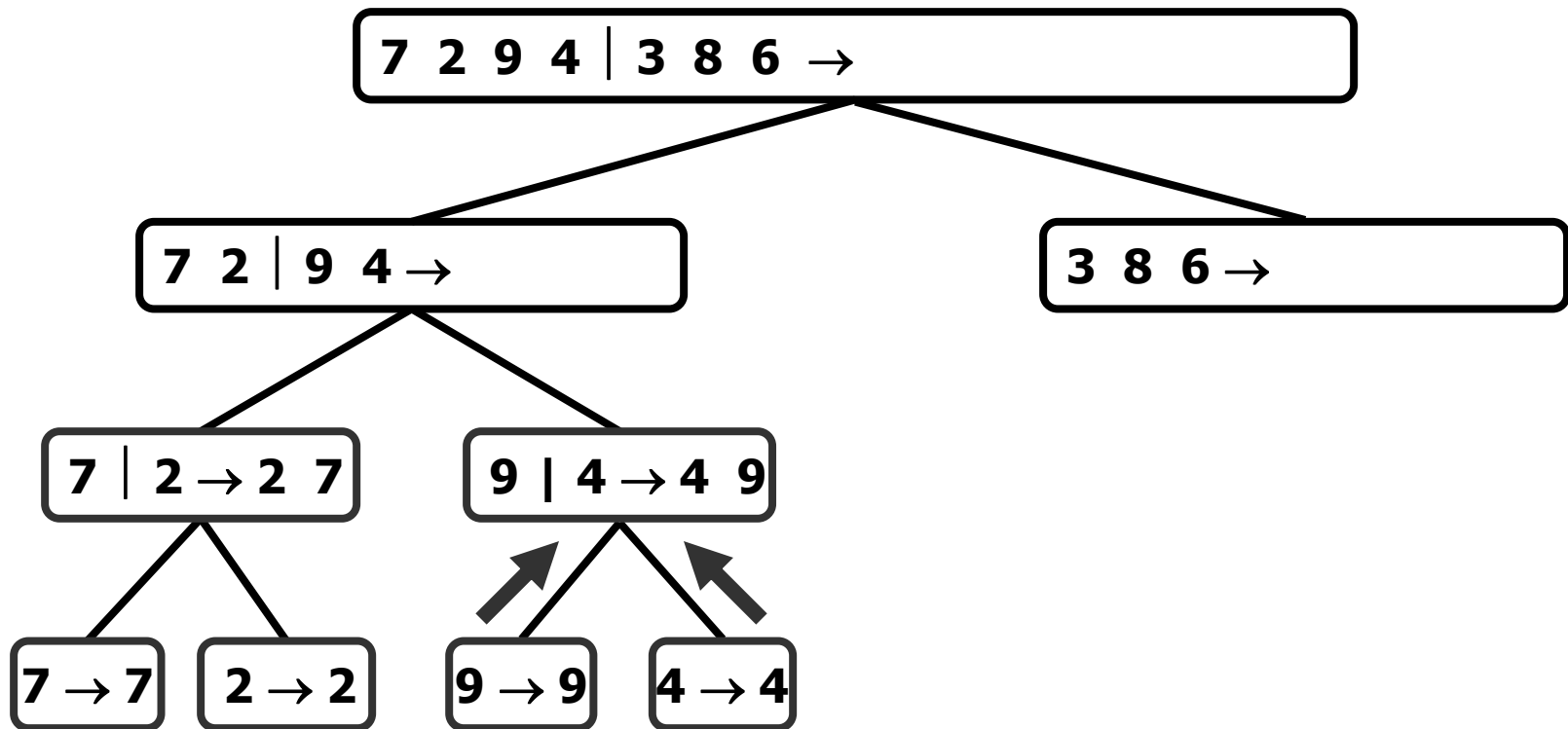
Merge-Sort Tree Example (cont.)

◆ Recursive right, divide



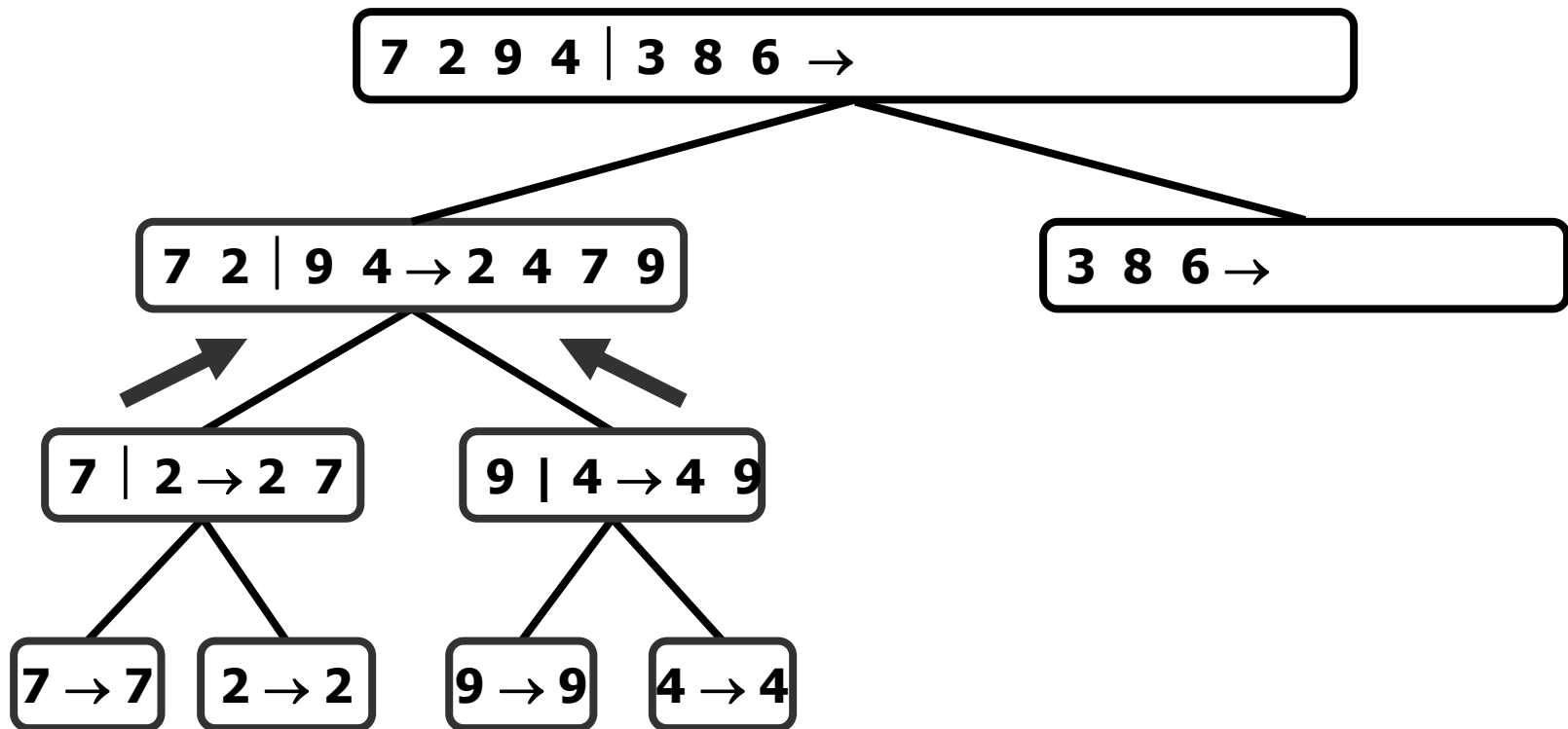
Merge-Sort Tree Example (cont.)

◆ Recursive left, ..., merge



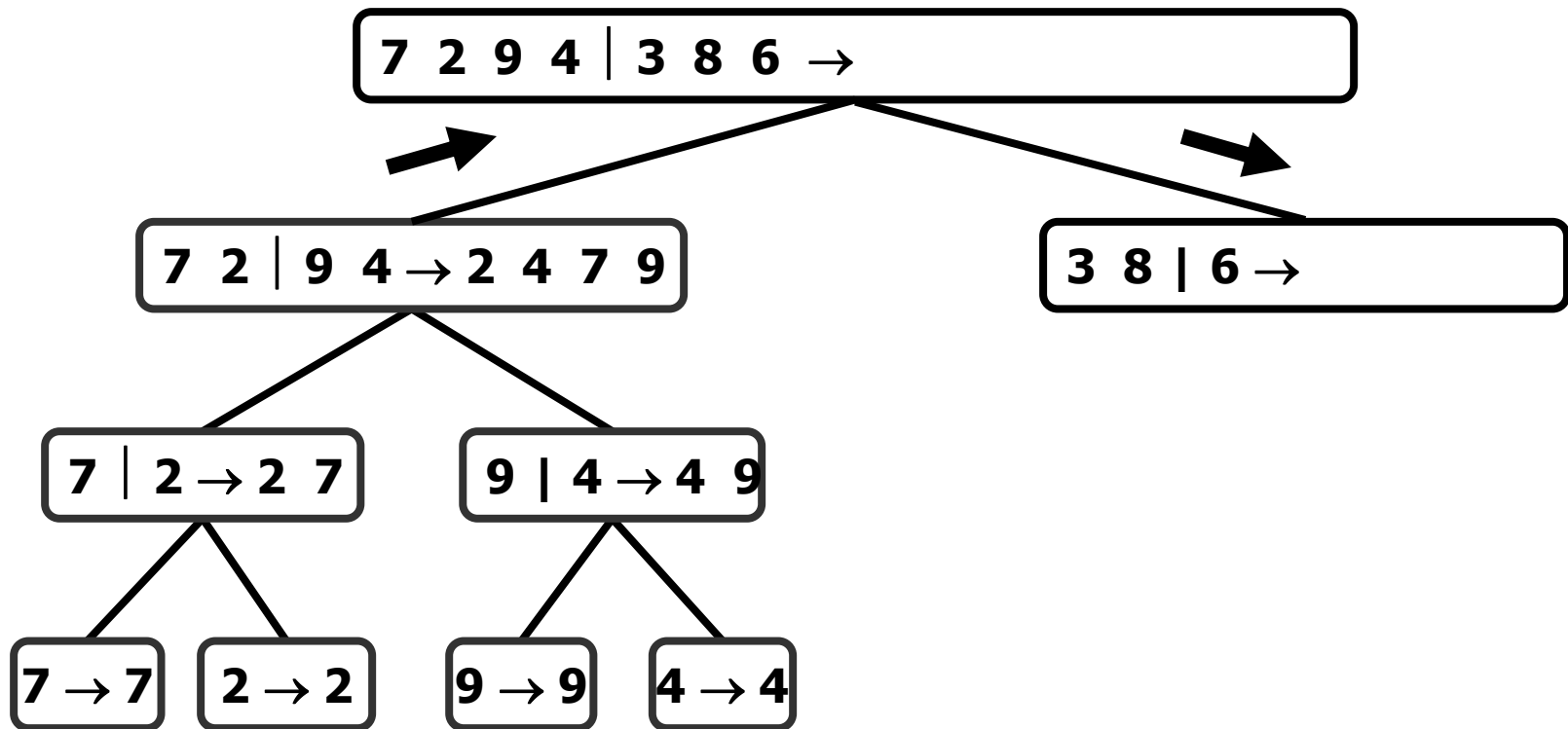
Merge-Sort Tree Example (cont.)

◆ Merge



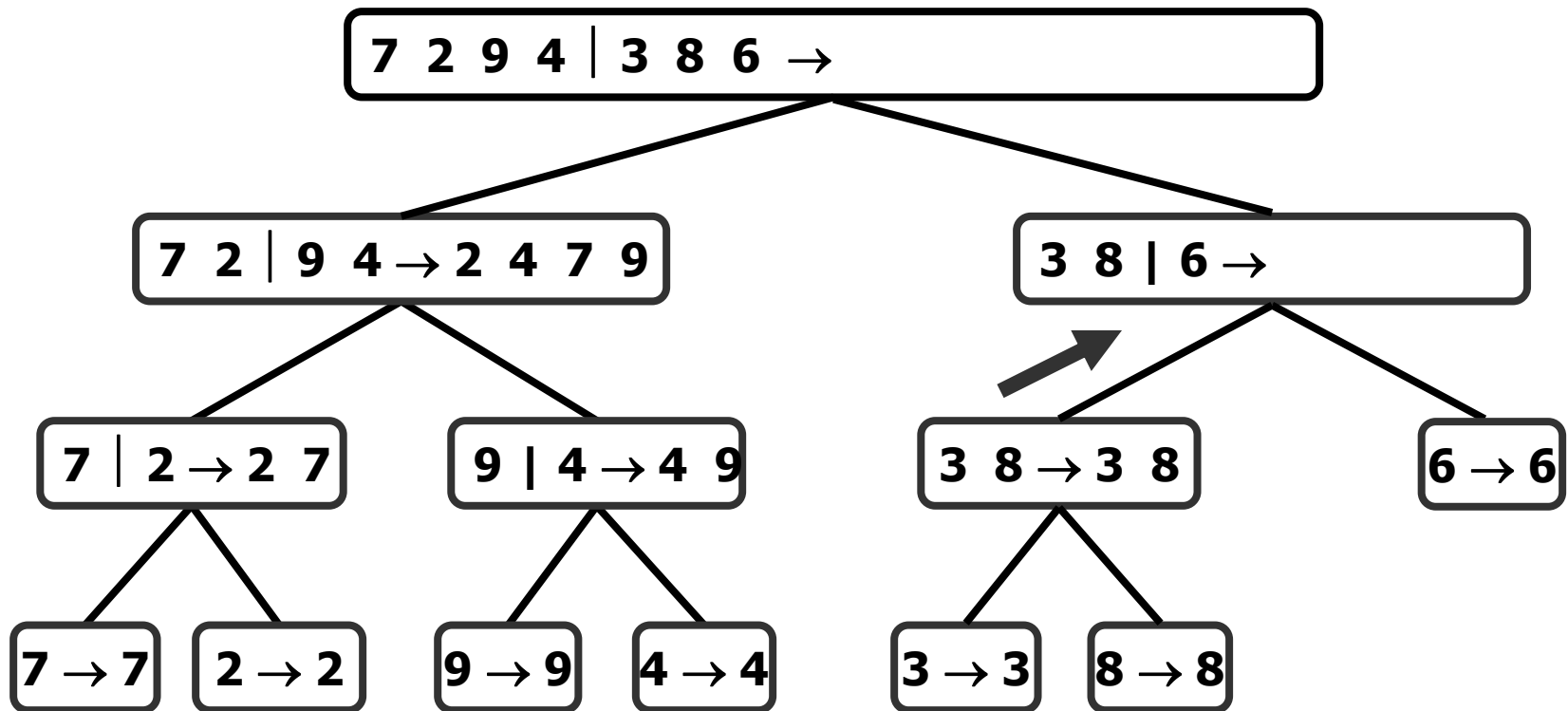
Merge-Sort Tree Example (cont.)

◆ Recursive right, divide



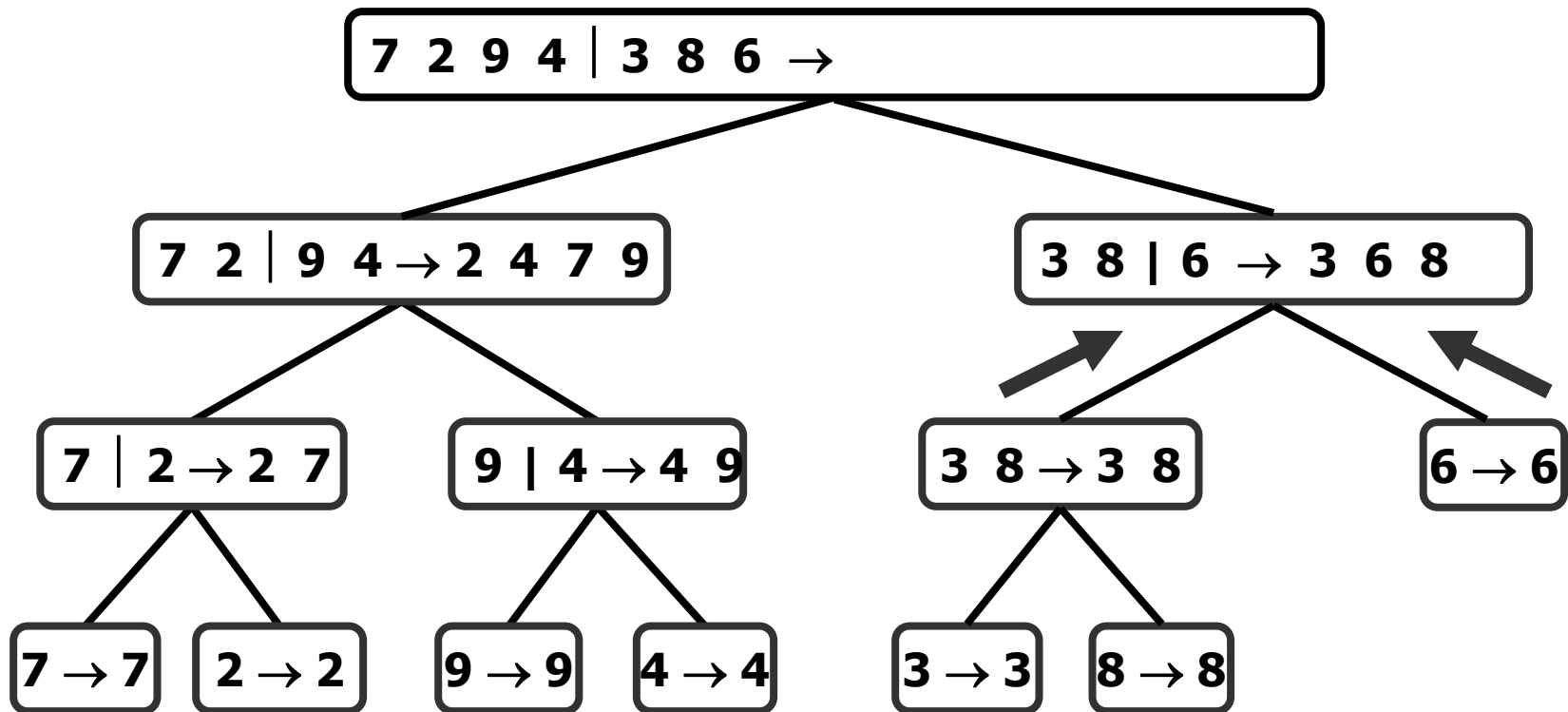
Merge-Sort Tree Example (cont.)

◆ Recursive left, ..., merge



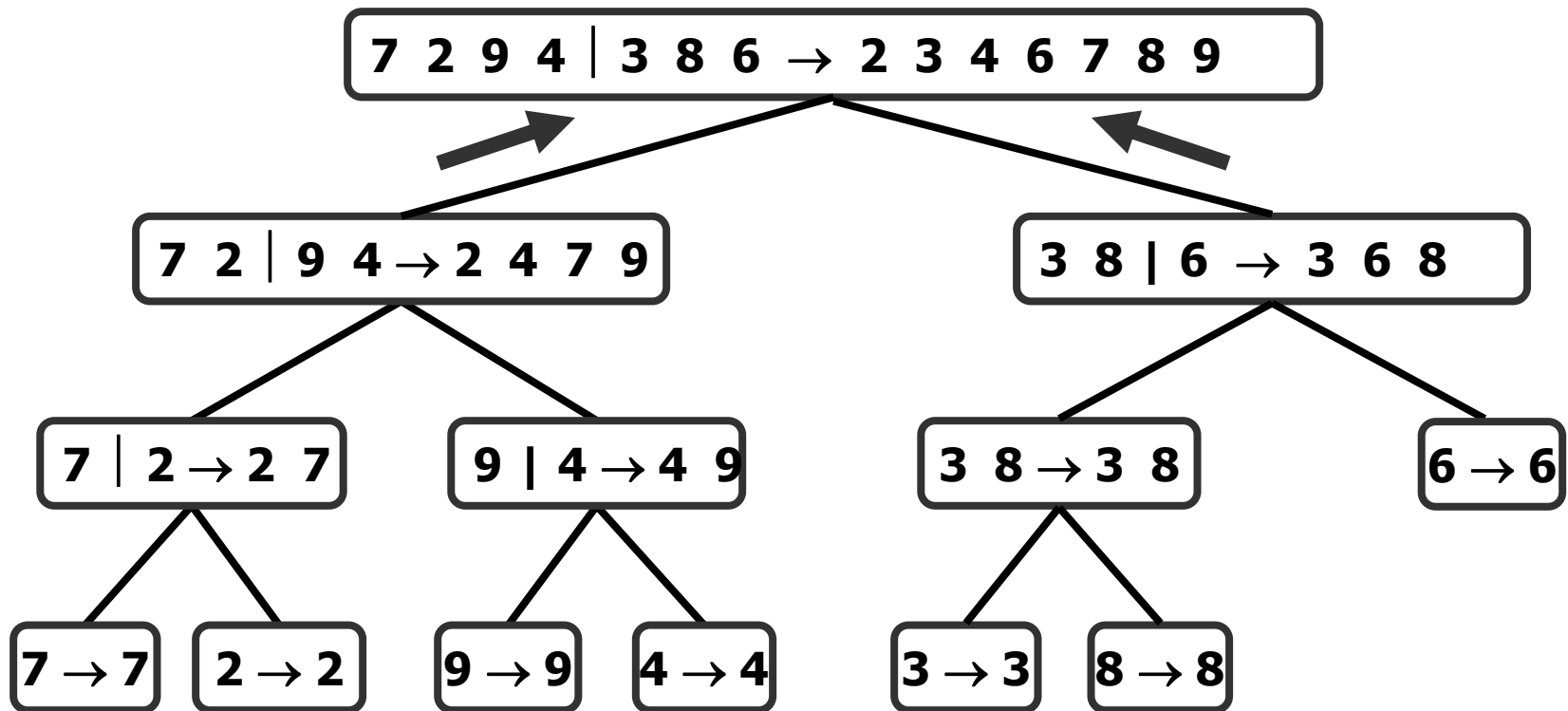
Merge-Sort Tree Example (cont.)

◆ Recursive right, base case, merge



Merge-Sort Tree Example (cont.)

◆ Merge



Array Output

- ◆ The previous Merge-Sort tree and the following array intermediate results are equivalent.

1. 7 2 9 4 3 8 6

2. 2 7 9 4 3 8 6

3. 2 7 4 9 3 8 6

4. 2 4 7 9 3 8 6

5. 2 4 7 9 3 8 6

6. 2 4 7 9 3 6 8

7. 2 3 4 6 7 8 9

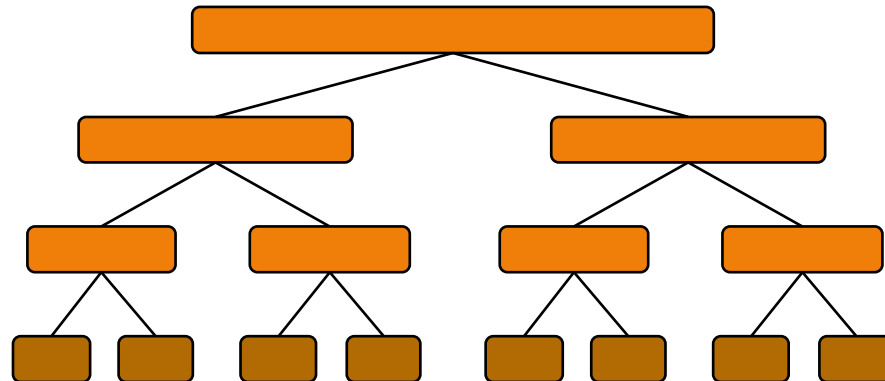
Analysis of Merge-Sort

- ◆ The height h of the merge-sort tree is $O(\log n)$
 - at each recursive call we divide in half the sequence,
- ◆ The overall amount of work done at the nodes of depth i is $O(n)$
 - we partition and merge 2^i sequences of size $n/2^i$
 - we make 2^{i+1} recursive calls
- ◆ Thus, the total running time of merge-sort is $O(n \log n)$

depth #seqs size

$$0 \qquad 1 \qquad n$$
$$1 \qquad 2 \qquad \mathbf{n/2}$$
$$i \quad 2^i \quad n/2^i$$

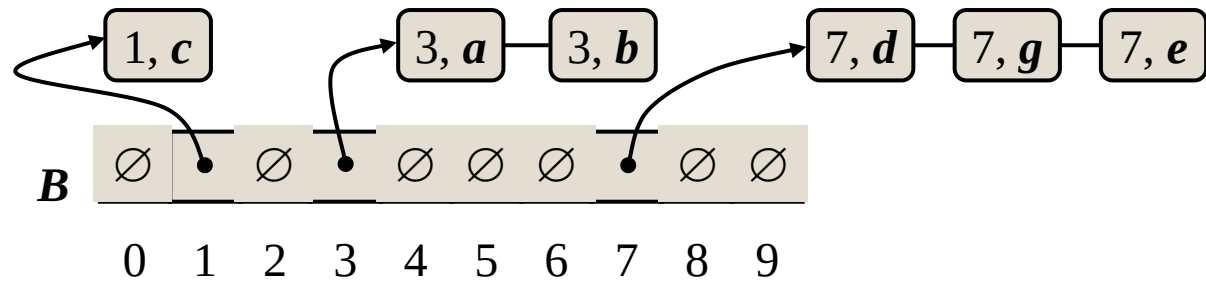
• • • • •



Summary of Sorting Algorithms

Algorithm	Time	Notes
selection-sort	$O(n^2)$	◆ in-place, unstable ◆ slow (good for small inputs)
quick-sort	$O(n \log n)$ $O(n^2)$ for worst case if not randomized	◆ in-place, unstable ◆ fastest (good for large inputs)
heap-sort	$O(n \log n)$	◆ in-place, unstable ◆ fast (good for large inputs)
merge-sort	$O(n \log n)$	◆ not in-place, stable ◆ sequential data access ◆ fast (good for huge inputs)

Bucket-Sort and Radix-Sort





Bucket-Sort (4.5.1)

- ◆ Let be S be a sequence of n (key, element) items with keys in the range $[0, N]$
- ◆ Bucket-sort uses the keys as indices into an auxiliary array B of sequences (buckets)

Phase 1: Empty sequence S by moving each item (k, o) into its bucket $B[k]$

Phase 2: For $i = 0, \dots, N$, move the items of bucket $B[i]$ to the end of sequence S

- ◆ Analysis:
 - Phase 1 takes $O(n)$ time
 - Phase 2 takes $O(n + N)$ time
- Bucket-sort takes $O(n + N)$ time

```
Algorithm bucketSort(S, N)
  Input  sequence  $S$  of
         (key  $k$ , element  $e$ ) items with
         keys in the range  $[0, N]$ 
  Output sequence  $S$  sorted by
         increasing keys

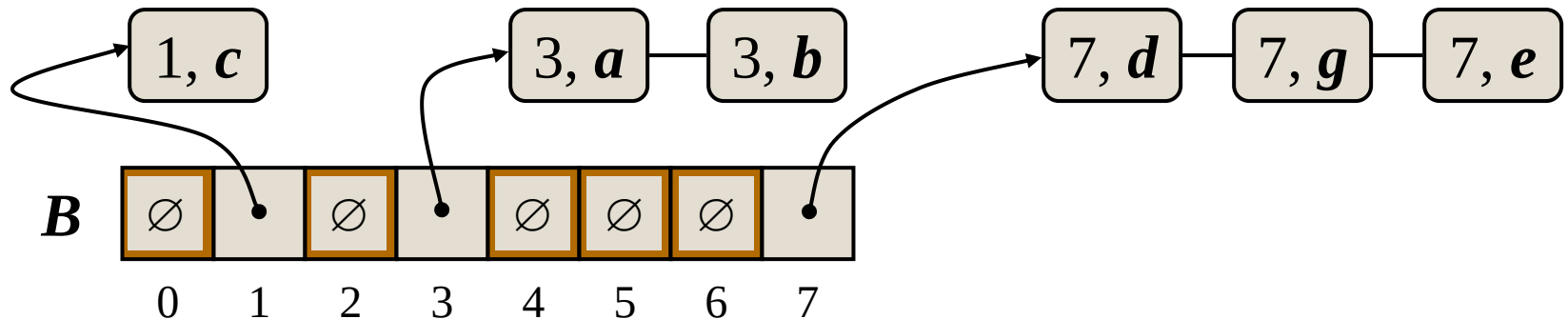
   $B$  = array of  $N$  empty sequences
  while not  $S.isEmpty()$ 
     $(k, e) = S.removeFirst()$ 
     $B[k].insertLast((k, e))$ 
  for  $i = 0$  to  $N$ 
    while not  $B[i].isEmpty()$ 
       $(k, e) = B[i].removeFirst()$ 
       $S.insertLast((k, e))$ 
```

Example



◆ Key range $[0, 7]$

Phase 1

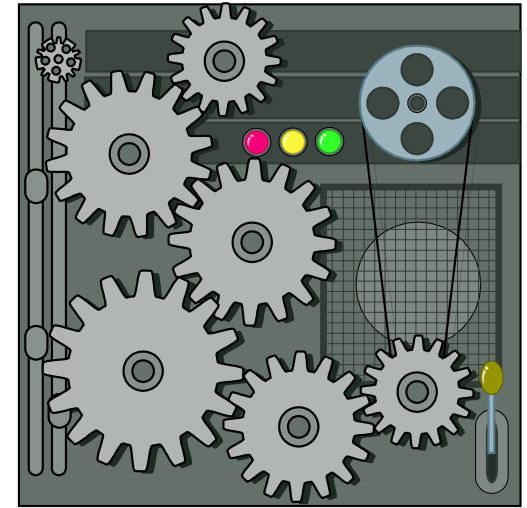


Phase 2



Radix-Sort (4.5.2)

- ◆ Radix-sort is a specialization of lexicographic-sort that uses bucket-sort as the stable sorting algorithm in each dimension
- ◆ Radix-sort is applicable to tuples where the keys in each dimension i are integers in the range $[0, N]$
- ◆ Radix-sort runs in time $O(d(n + N))$



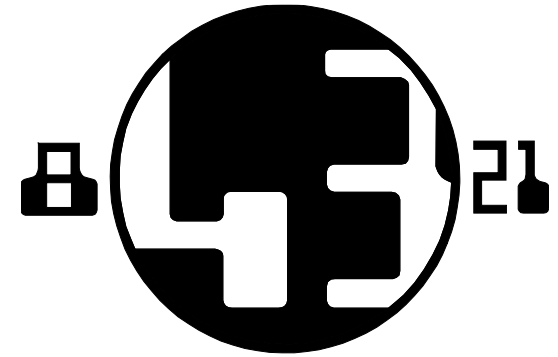
Algorithm radixSort(S , N)

Input: sequence S of d -tuples such that $(0, \dots, 0) \leq (x_d, \dots, x_0)$ and $(x_d, \dots, x_0) \leq (N, \dots, N)$ for each tuple $(x_{d-1}, \dots, x_0)$ in S

Output: sequence S sorted in lexicographic order

```
for  $i = 0$  to  $d-1$ 
    bucketSort( $S$ ,  $N$ ,  $i$ )
```

Radix-Sort for Binary Numbers



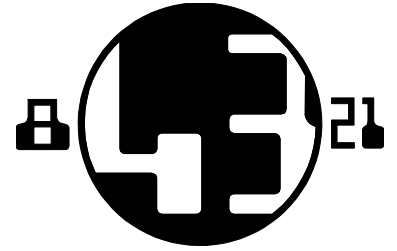
- ◆ Consider a sequence of n b -bit integers

$$x = x_{b-1} \dots x_1 x_0$$

- ◆ We represent each element as a b -tuple of integers in the range $[0, 1]$ and apply radix-sort with $N = 1$
- ◆ This application of the radix-sort algorithm runs in $O(bn)$ time
- ◆ For example, we can sort a sequence of 32-bit integers in linear time

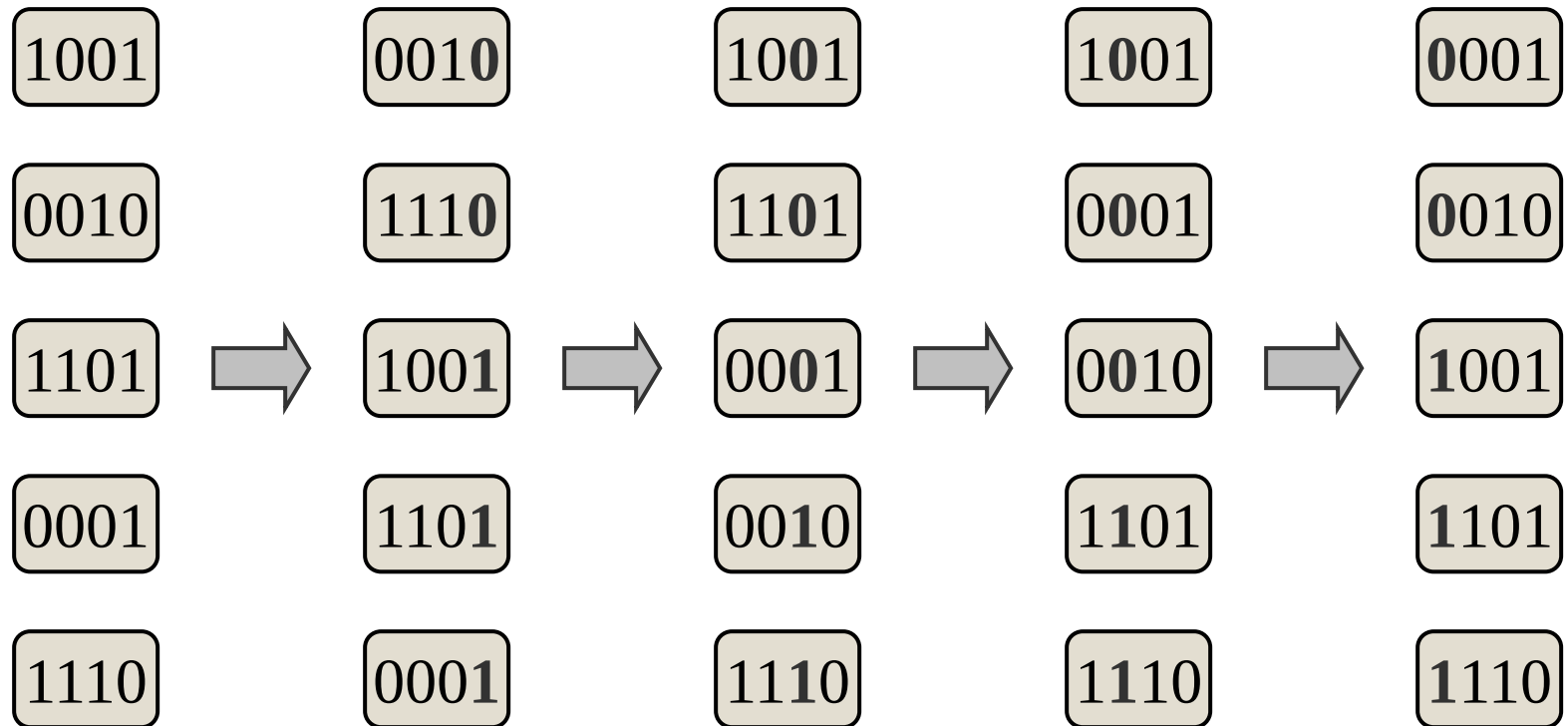
```
Algorithm binaryRadixSort(S)
Input: sequence  $S$  of  $b$ -bit integers
Output: sequence  $S$  sorted

replace each element  $x$ 
of  $S$  with item  $(0, x)$ 
for  $i = 0$  to  $b - 1$ 
    replace the key  $k$  of
    each item  $(k, x)$  of  $S$ 
    with bit  $x_i$  of  $x$ 
bucketSort( $S, 1$ )
```



Example

◆ Sorting a sequence of 4-bit integers





The Selection Problem

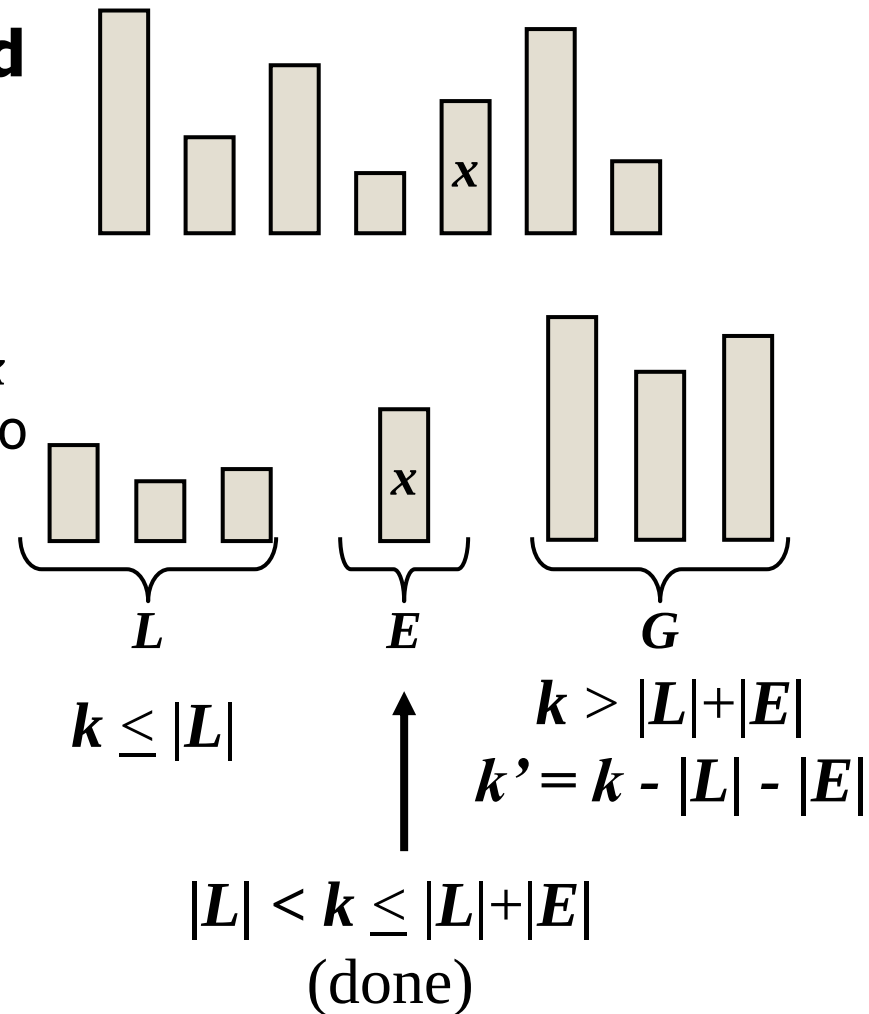
- ◆ Given an integer k and n elements $x_1, x_2, \dots, x_n$, taken from a total order, find the k -th smallest element in this set.
 - Smallest, $k = 0$
- ◆ Of course, we can sort the set in $O(n \log n)$ time and then index the k -th element.

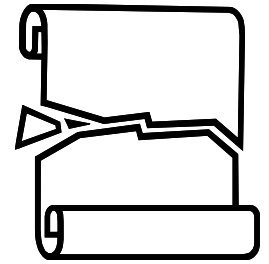
$k=2$ (3rd smallest) 77 44 99 22 66 → 22 44 **66** 77 99
- ◆ Can we solve the selection problem faster? Let's say $O(n)$.

Quick-Select (§ 4.7)

◆ Quick-select is a **randomized** selection algorithm based on the prune-and-search paradigm:

- Prune: pick a random element x (called pivot) and partition S into
 - ◆ L elements less than x
 - ◆ E elements equal x
 - ◆ G elements greater than x
- Search: depending on k , either answer is in E , or we need to recurse in either L or G





Algorithm Quick-Select

Algorithm QuickSelect (S, left, right, k)

Input: array S, left index, right index,
target smallest k index.

Output: S[k] where k is pi.

pi = Partition (S, left, right) // pivot index

if (k == pi)

return S[k]

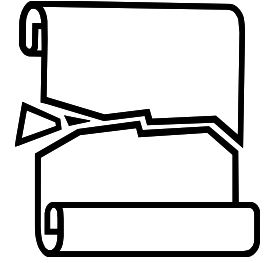
else if k < pi

return QuickSelect (S, left, pi-1, k)

else

return QuickSelect (S, pi+1, right, k)

Partition



- ◆ The partition step of Quick-Select is the same partition in Quick-Sort which takes $O(n)$ time.
- ◆ Based on Probabilistic Facts, Quick-Select runs in time $O(n)$.
- ◆ Quick-Select has the same worst case as Quick-Sort which takes $O(n^2)$ time.

Quick-Select Visualization

- ◆ Find 5th smallest number, hence $k = 4$
- ◆ In this example, pivot is always the last element.

$S = (77\ 99\ 22\ 66\ 55\ 44\ 11\ 88\ \underline{33})$, call partition

$pi = 2$, $S = (\underline{22}\ 11\ 33\ 77\ 99\ 66\ 55\ 44\ \underline{88})$, $k > pi$, call partition on right

$pi = 7$, $S = (\underline{22}\ 11\ 33\ 77\ 66\ 55\ \underline{44}\ 88\ 99)$, $k < pi$, call partition on left

$pi = 3$, $S = (\underline{22}\ 11\ 33\ 44\ 77\ 66\ \underline{55}\ 88\ 99)$, $k > pi$, call partition on right

$pi = 4$, $S = (\underline{22}\ 11\ 33\ 44\ \mathbf{55}\ 77\ 66\ 88\ 99)$, $k = pi$, stop

5th smallest = 55